МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

КАФЕДРА №43

КУРСОВОЙ ПРОЕКТ
ЗАЩИЩЕН С ОЦЕНКОЙ

РУКОВОДИТЕЛЬ

канд. техн. наук		А.А. Попов
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА К КУРСОВОЙ РАБОТЕ

на тему: ”Сайт по аренде жилья”

по дисциплине: ПРОГРАММИРОВАНИЕ ВСТРОЕННЫХ ПРИЛОЖЕНИЙ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4236		А.В. Панченко
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2025

СОДЕРЖАНИЕ

1. ЗАДАНИЕ НА КУРСОВОЕ ПРОЕКТИРОВАНИЕ 3
1.1. Назначение разрабатываемого прототипа 3
1.2. Режимы работы разрабатываемого прототипа 4
2. ТЕХНИЧЕСКОЕ ЗАДАНИЕ НА ПРОТОТИП 6
2.1. Список требований к системе 6
2.2. Описание форматов 8
2.3. Диаграмма вариантов использования 9
2.4. Список функций, возлагаемых на прототип 9
3. ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ СИСТЕМЫ 10
3.1. Контекстная диаграмма системы 10
3.2. Структурная схема модулей и соединений 11
3.3. Диапазоны напряжений и протоколы 11
3.4. Архитектурная схема и взаимодействие элементов 12
3.5. Принципы взаимодействия элементов 12
4. РАЗРАБОТКА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ СИСТЕМЫ 15
ЗАКЛЮЧЕНИЕ 33
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 34
ПРИЛОЖЕНИЕ А 35

1.ЗАДАНИЕ НА КУРСОВОЕ ПРОЕКТИРОВАНИЕ

1.1. Назначение разрабатываемого прототипа

Разрабатываемый прототип представляет собой веб-приложение для аренды жилья, предназначенное для облегчения взаимодействия между арендаторами и арендодателями. Основное назначение прототипа заключается в создании функциональной платформы, которая обеспечивает:

- Регистрацию и аутентификацию пользователей с разделением ролей (арендатор и арендодатель).
- Создание, просмотр, редактирование и удаление объявлений о сдаче жилья в аренду, включая загрузку фотографий и интеграцию с Яндекс.Картами для отображения расположения объектов и автозаполнения адресов через обратное геокодирование.
- Поиск и фильтрацию объявлений по различным критериям (цена, количество комнат, тип жилья, район, удобства).
- Организацию общения между пользователями через чаты в реальном времени с использованием Socket.IO, а также уведомления о новых сообщениях с возможностью отключения.
- Систему бронирований с проверкой доступности дат, мини-календарем занятых периодов, списками "Мои бронирования" для арендаторов и "Заказы" для арендодателей, включая подтверждение или отклонение бронирований.
- Механизм отзывов и рейтингов с оценками от 1 до 5, ограничением на один отзыв и расчетом среднего рейтинга.
- Поддержку избранного, истории просмотров объявлений.
- Мультиязычность (русский и английский) с переключателем и хранением предпочтений.
- Адаптацию интерфейса под мобильные устройства для удобства использования форм, списков, чатов и календаря.

Прототип моделирует реальную систему аренды жилья, демонстрируя применение стека технологий Node.js, Express, PostgreSQL, Socket.IO и

Яндекс.Карт для создания безопасного, масштабируемого и пользовательского приложения.

1.2. Режимы работы разрабатываемого прототипа

Прототип функционирует в клиент-серверной архитектуре с использованием HTTP/HTTPS для базовых операций и WebSocket для реального времени. Режимы работы зависят от статуса аутентификации и роли пользователя:

1. Режим гостевого доступа (неавторизованный пользователь)

- Просмотр главной страницы с списком объявлений и интегрированной картой (Яндекс.Карты).
- Поиск по заголовку и адресу, применение фильтров (цена, комнаты, тип жилья, район, удобства).
- Просмотр деталей объявлений без возможности взаимодействия (бронирование, чаты, добавление в избранное).
- Доступ к страницам регистрации и входа.

2. Режим авторизованного пользователя (общий)

- Аутентификация через email с хешированием паролей (bcryptjs) и сессиями.
- Доступ к личному кабинету: отображение email, display_name (с редактированием), управление уведомлениями.
- Навигация по страницам: объявления, чаты, избранное, история просмотров ("Вы недавно смотрели" — хранение локально или в БД).
- Реальное время: обмен сообщениями в чатах по объявлениям, уведомления о новых сообщениях.
- Отзывы: добавление оценок (1-5), просмотр среднего рейтинга с ограничением на один отзыв.
- Мультиязычность: переключение между русским и английским, хранение в cookie/localStorage.
- Адаптация: responsive дизайн для мобильных устройств.

3. Режим арендатора

- Бронирование: форма с проверкой дат на пересечения, мини-календарь, список "Мои бронирования" с статусами.
- Добавление в избранное, инициирование чатов с арендодателями.
- Добавление отзывов о арендодателях после бронирования.
- Просмотр истории просмотров.

4. Режим арендодателя

- Управление объявлениями: создание/редактирование/удаление с данными (адрес, район, комнаты, тип, удобства), загрузкой фото (multer), геокодированием.
- Обработка заказов: список "Заказы", подтверждение/отклонение бронирований.
- Ответы в чатах на запросы арендаторов.
- Добавление отзывов об арендаторах.

Прототип обеспечивает безопасность данных, оптимизацию для реального времени и интеграцию с базой данных PostgreSQL (таблицы: users, listings, messages, reviews, favorites, bookings).

2.ТЕХНИЧЕСКОЕ ЗАДАНИЕ НА ПРОТОТИП

2.1.Список требований к системе

В соответствии со спецификацией Software Requirements Specification (SRS) представлен список требований к системе в таблице по разделам.

Раздел	**Требования**
Требования к реализации (язык программирования и библиотеки)	Язык программирования: JavaScript (Node.js для backend). Фреймворк: Express. База данных: PostgreSQL (PSQL). Библиотеки: bcryptjs (хеширование паролей), multer (загрузка фотографий), Socket.IO (реальное время для чатов и уведомлений), Яндекс.Карты (интеграция карт и геокодирование). Сессии для управления доступом.
Функциональные требования	Аутентификация: регистрация/вход по email, хеширование паролей, сессии, роли (арендатор/арендодатель), редактирование display_name. Структура БД: таблицы users, listings, messages, reviews, favorites, bookings. Личный кабинет: отображение email и имени, добавление объявлений (для арендодателя). Навигация: главная (список объявлений и карта), логин/регистрация, профиль, страницы объявлений (список, просмотр, создание, редактирование, удаление), чаты (список диалогов,

Раздел	Требования
	страница диалога), избранное, "Мои бронирования" (арендатор), "Заказы" (арендодатель). Объявления: загрузка фото, данные (адрес, район, комнаты, тип, удобства), интеграция Яндекс.Карт, обратное геокодирование. Поиск и фильтры: по заголовку/адресу, фильтры (цена, комнаты, тип, район, удобства). Чаты и уведомления: Socket.IO для реального времени, диалоги по объявлениям, уведомления о сообщениях с отключением. Отзывы: оценки 1-5, один отзыв per пользователь, средний рейтинг. Бронирования: форма, проверка дат, мини-календарь, списки бронирований, подтверждение/отклонение. Мультиязычность: русский/английский, переключатель, хранение в cookie/localStorage. Мобильная адаптация: формы, списки, чат, календарь. История просмотров: сохранение (локально/в БД), блок "Вы недавно смотрели".
Аппаратные требования	Сервер: стандартный сервер с Node.js (CPU: минимум 2 ядра, RAM: минимум 4 GB, диск: минимум 50

Раздел	Требования
	GB). Клиент: веб-браузер (Chrome, Firefox и т.п.) на ПК/мобильном устройстве с поддержкой JavaScript, WebSocket и HTML5. Нет специальных аппаратных требований для приборов, так как это веб-приложение.

Таблица 1 – Таблица требований

2.2.Описание форматов

- **Форматы проектируемых команд**: API-эндпоинты на Express (HTTP методы: GET для просмотра, POST для создания/регистрации/сообщений, PUT для редактирования, DELETE для удаления). Примеры: /api/register (POST с JSON {email, password, role}), /api/login (POST с JSON {email, password}), /api/listings (GET для списка, POST для создания с multipart/form-data для фото).
- **Форматы отображения информации**: HTML-страницы с CSS для адаптации (responsive design). Карточки объявлений: заголовок, фото, адрес, цена. Карта: Яндекс.Карты с маркерами. Чаты: список диалогов (имя собеседника, последнее сообщение), страница диалога (список сообщений с timestamp). Календарь: мини-календарь с выделением занятых дат. Отзывы: звезды 1-5, текст.
- **Формат обмена пакетами данных (транспортный уровень)**: HTTP/HTTPS для REST API (JSON для данных, multipart для файлов). WebSocket (Socket.IO) для реального времени: события (message, notification) с JSON {user_id, listing_id, text, timestamp}.
- **Форматы сигналов**: Не применимо (веб-приложение, нет аппаратных сигналов).
- **Форматы кнопок и т.п.**: UI-элементы: кнопки (HTML <button>) для действий (вход, регистрация, отправка сообщения, бронирование,

подтверждение). Формы: с input для email/password/адрес/фильтры. Переключатель языка: dropdown или toggle-button.

2.3.Диаграмма вариантов использования

Текстовая диаграмма use case (в формате Mermaid для представления):

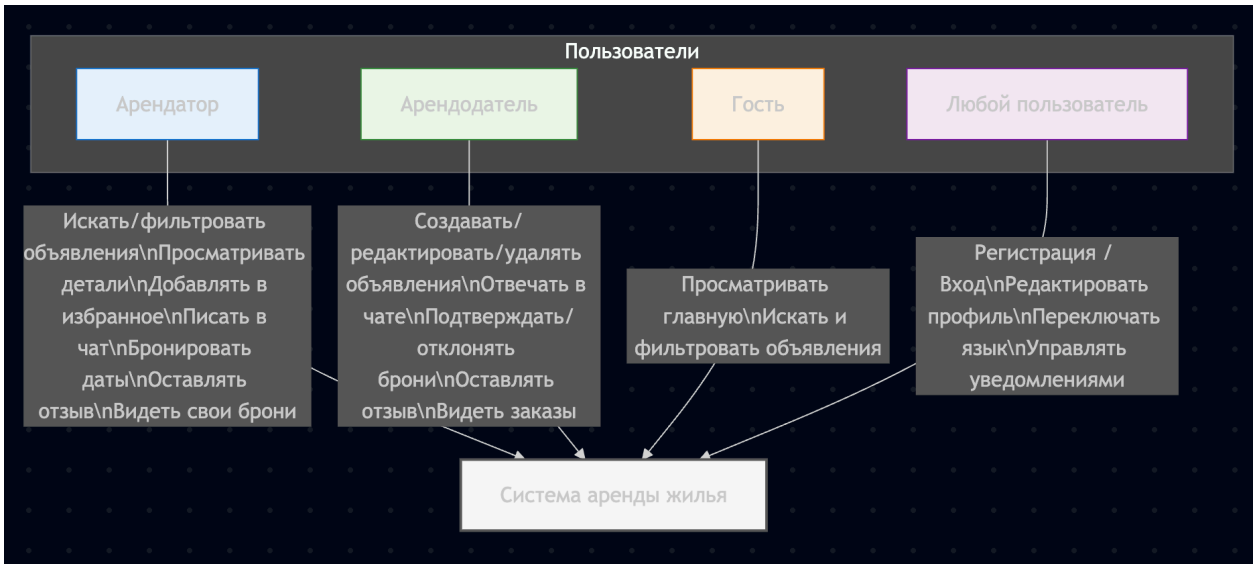

Рисунок 1 – Диаграмма use case

2.4.Список функций, возлагаемых на прототип

- Аутентификация и управление учетными записями (регистрация, вход, роли, редактирование display_name).
- Управление базой данных (таблицы users, listings, messages, reviews, favorites, bookings).
- Отображение и управление личным кабинетом.
- Навигация по страницам (главная, объявления, чаты, избранное, бронирования, заказы).
- Создание и управление объявлениями (данные, фото, карта, геокодирование).
- Поиск и фильтрация объявлений.
- Чаты и уведомления в реальном времени.
- Система отзывов и рейтингов.
- Управление бронированиями (форма, проверка дат, календарь, подтверждение).
- Мультиязычность и хранение предпочтений.

- Мобильная адаптация интерфейса.
- История просмотров объявлений.

3.ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ СИСТЕМЫ

3.1.Контекстная диаграмма системы

Система "Renting-Housing" представляет собой веб-приложение для сдачи и аренды жилой недвижимости. На верхнем уровне система взаимодействует со следующими внешними компонентами:

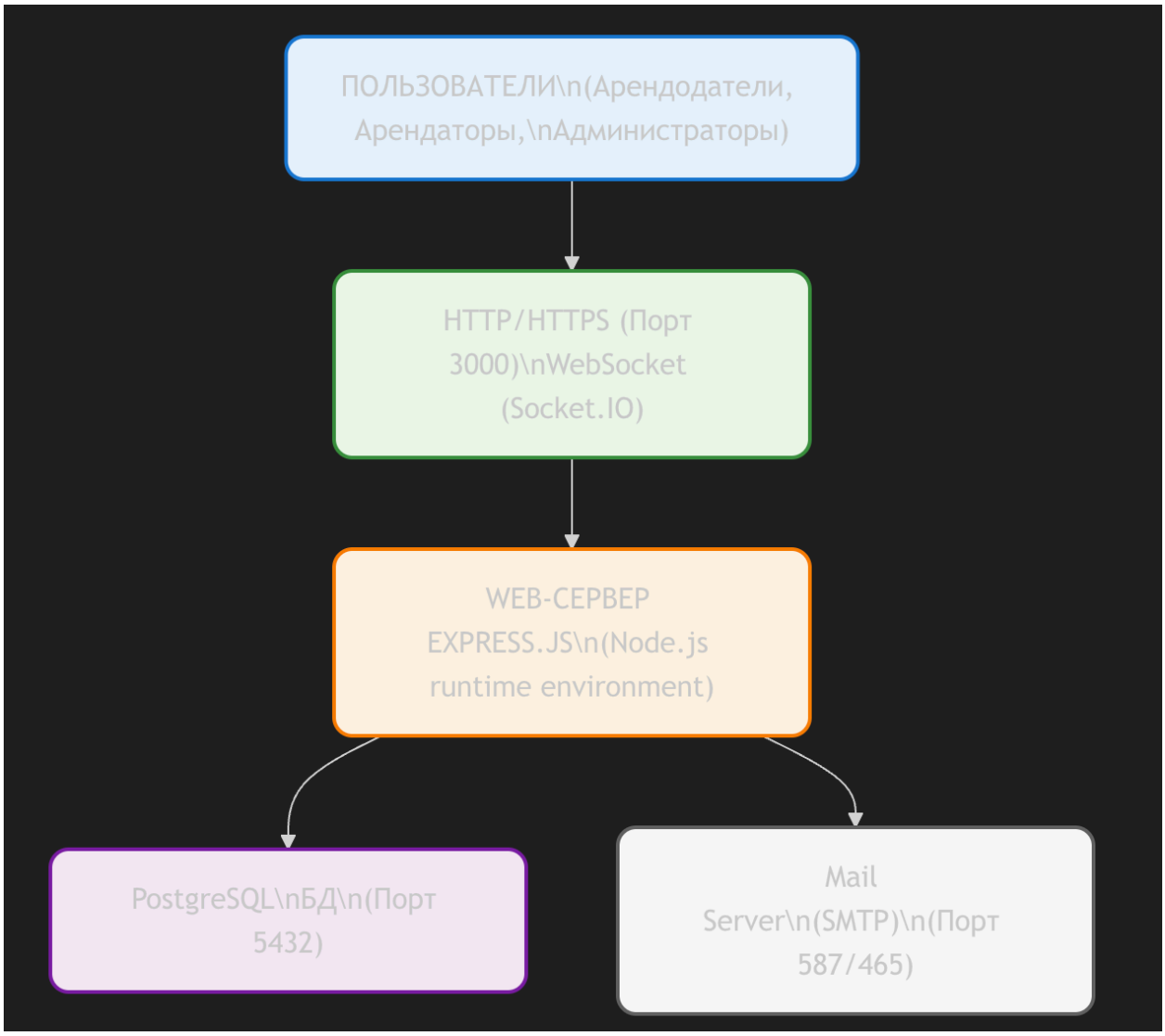

Рисунок 2 – Контекстная диаграмма системы

3.2.Структурная схема модулей и соединений

№	Компонент	Функция	Тип соединения	Порт/Протокол
1	Express.js	HTTP-сервер	TCP/IP	3000 (HTTP)
2	Socket.IO	Real-time коммуникация	WebSocket	3000 (WS)
3	PostgreSQL Pool	Управление БД	TCP/IP	5432
4	Nodemailer	Email уведомления	SMTP	587/465
5	Multer	Загрузка файлов	HTTP multipart	3000
6	bcryptjs	Шифрование паролей	In-process	N/A
7	Express-session	Управление сессиями	HTTP Cookies	3000

Таблица 2 – Схема модулей соединения

3.3.Диапазоны напряжений и протоколы

TCP/IP Stack (Application Layer):

- **HTTP Protocol**:
  - Формат: текст (ASCII)
  - Скорость: Зависит от пропускной способности сети (обычно до 1 Gbps)
  - Порт: 3000
- **WebSocket Protocol**:
  - Формат: JSON (текстовый)
  - Скорость: Real-time (задержка <100ms рекомендуется)

- Порт: 3000
- Назначение: Мгновенные уведомления, чат

- **SMTP Protocol**:
  - Формат: текст (RFC 5321)
  - Скорость: асинхронная (без ограничения задержки)
  - Порт: 587 (TLS) или 465 (SSL/TLS)
  - Назначение: Отправка email-уведомлений
- **PostgreSQL Protocol**:
  - Формат: бинарный протокол PostgreSQL
  - Скорость: высокоскоростная локальная сеть (0-10ms локально)
  - Порт: 5432
  - Назначение: Хранение и запрос данных

3.4.Архитектурная схема и взаимодействие элементов

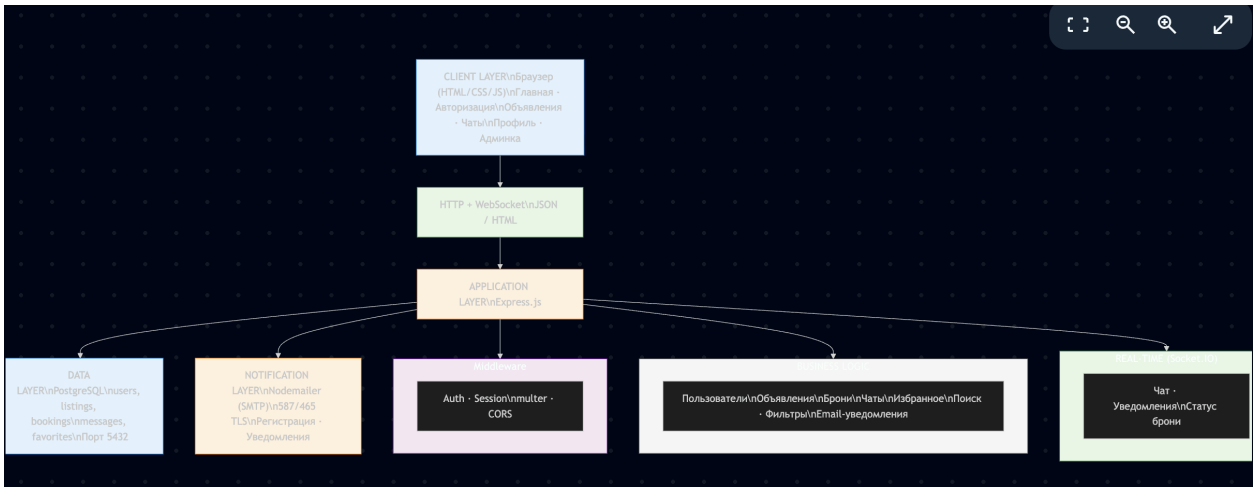

Рисунок 3 – Архитектура элементов

3.5.Принципы взаимодействия элементов

```
Пользователь → Browser → HTTP POST /login → Express.js → bcryptjs (хеширование)
          ↓
  Проверка в PostgreSQL (users table) → Express-session (создание сессии)
          ↓
  Установка Cookie → Browser → Redirect на главную
```

Рисунок 4 – Поток данных при аутентификации

```
Пользователь заполняет форму → Multer (загрузка файлов) → Express.js
          ↓
  Валидация данных → PostgreSQL INSERT в таблицу listings
          ↓
  Socket.IO emit → Уведомление всем пользователям → Browser обновление списка
```

Рисунок 5 – Поток данных при публикации объявления

```
Отправитель набирает сообщение → WebSocket emit → Express.js + Socket.IO
          ↓
  PostgreSQL INSERT в таблицу messages → Socket.IO emit к получателю
          ↓
  Browser получает обновление → Real-time отображение в чате
          ↓
  Параллельно: Email уведомление через SMTP (Nodemailer)
```

Рисунок 6 – Поток данных в системе чата

Инфраструктура хранения:

Таблица БД	Назначение	Основные поля	Связи
users	Профили пользователей	id, email, password_hash, name, phone	1:M с listings, bookings
listings	Объявления об аренде	id, user_id, title, price, address, photos, amenities, lat, lng	M:1 с users, 1:M с bookings
bookings	Бронирования	id, user_id, listing_id, check_in, check_out, status	M:1 с users и listings
messages	Сообщения между пользователями	id, sender_id, receiver_id, text, is_read, created_at	M:1 с users
favorites	Избранные объявления	id, user_id, listing_id	M:1 с users и listings

Таблица 3 – Инфраструктура хранения

Характеристики протоколов и сигналов:

HTTP/1.1 & HTTP/2:

- Формат: текстовый (ASCII + бинарные данные в multipart)
- Кодирование: UTF-8

- Скорость: Зависит от сетевого соединения (обычно 10-100 Mbps)
- Задержка: 10-500ms

WebSocket (Socket.IO):

- Формат: JSON объекты
- Кодирование: UTF-8 + бинарные фреймы
- Задержка: <100ms (Real-time)
- Масштабируемость: до 10,000+ соединений на одном сервере

PostgreSQL Protocol:

- Формат: бинарный запрос-ответ
- Скорость запроса: 1-50ms (локально)
- Кодирование: PgSQL native format
- Поддержка: транзакции, ACID-compliance

SMTP Protocol (Email):

- Формат: текстовый (RFC 5321)
- Кодирование: UTF-8 (с Base64 для вложений)
- Задержка: 1-5 секунд (асинхронно)
- Надежность: Retry mechanism

Безопасность и масштабируемость:

- **Аутентификация**: Session-based (express-session) + bcryptjs хеширование
- **Транспортная безопасность**: HTTPS (рекомендуется в production)
- **Защита БД**: Parameterized queries (pg driver)
- **Загрузка файлов**: Валидация типов через multer
- **Real-time обновления**: Socket.IO с аутентификацией

4.РАЗРАБОТКА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ СИСТЕМЫ

Контекст системы

Система — веб-приложение Renting-Housing (Node.js, Express, Socket.IO) для публикации объявлений, бронирований и чата. Внешние сущности: пользователи (арендодатели, арендаторы, администратор), SMTP-сервер (почта), PostgreSQL (хранилище), веб-браузер клиента. Клиент взаимодействует через HTTP/HTTPS и WebSocket.

Компоненты системы (кратко, по коду app.js)

- Веб-сервер: Express.js (HTTP маршруты, middleware).
- Реальное время: Socket.IO на том же HTTP сервере (WebSocket/Fallback).
- СУБД: PostgreSQL через pg.Pool (порт 5432).
- Почта: Nodemailer (при наличии env переменных: SMTP_HOST/USER/PASS).
- Загрузка файлов: Multer (multipart/form-data).
- Сессии: express-session (куки).
- Хеширование паролей: bcryptjs.
- Вспомогательные функции: notifyUser(io, userId, payload), emailUser(userId,...), ensureListingExtraColumns().

Аппаратно-протокольные характеристики (применимо к web-архитектуре)

- Уровень физ/электрический: стандартные Ethernet/Wi-Fi интерфейсы в пределах 3.3V/5V.
- Транспорт/Сеть: TCP/IP (порт 3000 локально для dev), HTTP/1.1 или HTTP/2 — текстовый формат запросов/ответов (UTF-8, JSON в теле), скорость зависит от сети (100 Mbps — 1 Gbps типично для LAN).
- Реальное время: WebSocket (Socket.IO) — обмен JSON объектами по WebSocket; низкая задержка (<100 ms) требуется для чата.

- Почта: SMTP (порт 587 — STARTTLS, или 465 — SSL/TLS), текстовый протокол (RFC 5321), вложения в Base64.
- БД: собственный бинарный/текстовый протокол PostgreSQL, порт 5432, транзакции ACID.

Структурная схема соединений модулей (текстовое описание с номерами линий связи/портов)

Нумерация линий связи — логическая, для документации:

- Линия 1: Клиент (браузер) → Express HTTP маршруты. Порт: 3000 (dev). Протокол: HTTP/HTTPS (TCP 80/443 в production). Формат: JSON / form-data / HTML.
- Линия 2: Клиент ↔ Socket.IO (WebSocket). Порт: 3000. Формат: JSON события.
- Линия 3: Express → PostgreSQL (pg.Pool). Порт: 5432. Протокол: PostgreSQL binary/text.
- Линия 4: Express → SMTP (nodemailer transport). Порты: 587/465. Протокол: SMTP.
- Линия 5: Express (multer) ↔ файловая система/объектное хранилище (локально в images или в облаке). Тип: файловые операции (FS) или HTTP API облачного хранилища.

Схема архитектуры системы (логическая)

- Клиентская часть:
  статические страницы views/*.html + main.js отправляют запросы и подключаются по Socket.IO.
- Сервер: app.js — единственная точка входа, содержит маршруты, middleware, socket обработчики и вспомогательные функции.
- Хранилище данных: PostgreSQL
  (таблицы users, listings, bookings, messages, favorites, orders).
- Внешние сервисы: SMTP для уведомлений.

- Поток управления: HTTP API для CRUD операций и авторизации; WebSocket для real-time уведомлений/чата; асинхронные фоновые задачи (email) — выполняются в процессе сервера через nodemailer.

Организация взаимодействия элементов

- HTTP маршруты выполняют валидацию, аутентификацию (через сессии), доступ к БД через pg.Pool, возвращают HTML/JSON.
- WebSocket (Socket.IO) использует namespaces/rooms user_<id> для отправки уведомлений одному пользователю (io.to('user_123').emit(...)).
- При изменениях данных сервер эмитит соответствующие события по WebSocket (оповещения о новых объявлениях, новых сообщениях, изменении статуса брони). Одновременно при необходимости отправляется почта через Nodemailer.
- Все запросы к БД должны быть параметризованы (пул используется в коде).

Разработка ПО для элементов архитектуры (алгоритмы, диаграммы состояний, последовательности и интерфейсы)

1. Алгоритмы инициализации (граф-схема)

- Инициализация сервера (псевдо-алгоритм)
    1. Загружаются env переменные (SMTP_* и др.).
    2. Настраивается pg.Pool с параметрами подключения (user, host, database, password, port).
    3. Инициализируется express() и http.createServer(app); создаётся new Server(server) для Socket.IO.
    4. Настраивается middleware: session, body parsers, статические директории, multer для загрузки.
    5. Вызывается ensureListingExtraColumns() (ленивая инициализация колонок) — если необходимо, выполняются ALTER TABLE.
    6. Если SMTP env заполнены — создаётся nodemailer.createTransport(...).

7. Настраиваются Socket.IO handlers (on connection → аутентификация по сессии, join rooms user_<id>).
8. Запуск сервера: server.listen(port).

Псевдо-код (упрощённо):

```
loadEnv()
pool = new Pool(config)
app = express()
server = http.createServer(app)
io = new Server(server)
setupMiddleware(app)
if (!listingsColumnsEnsured) ensureListingExtraColumns()
if (SMTP configs exist) mailer = createTransport(...)
setupRoutes(app, pool, mailer, io)
setupSocketHandlers(io)
server.listen(PORT)
```

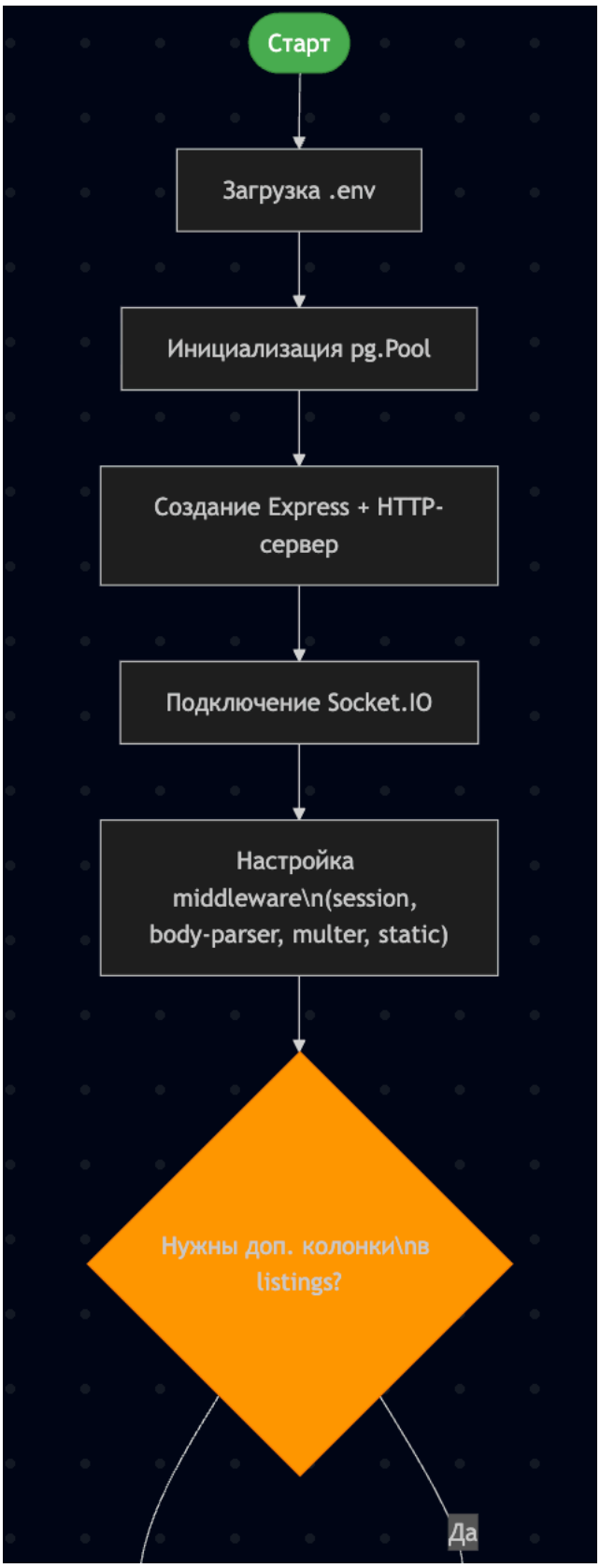

Рисунок 7 – Алгоритм инициализации 1

Рисунок 8 – Алгоритм инициализации 2

2. Алгоритмы работы ключевых модулей

- Аутентификация (вход пользователя)
    - Входной маршрут: POST /login

- Алгоритм:
    1. Получить email и password из тела запроса.
    2. Проверить формат email (isValidEmail).
    3. SELECT password_hash, id FROM users WHERE email = $1.
    4. Если пользователь найден — сравнить bcrypt.compareSync(password, hash).
    5. При успехе: req.session.userId = id; respond 200 с редиректом/JSON. Иначе — 401.

Псевдо:

```
POST /login:
  email, password = req.body
  if not isValidEmail(email): return 400
  row = pool.query("SELECT id, password_hash FROM users WHERE email=$1", [email])
  if row empty: return 401
  if bcrypt.compare(password, row.password_hash):
    req.session.userId = row.id
    return 200
  else
    return 401
```

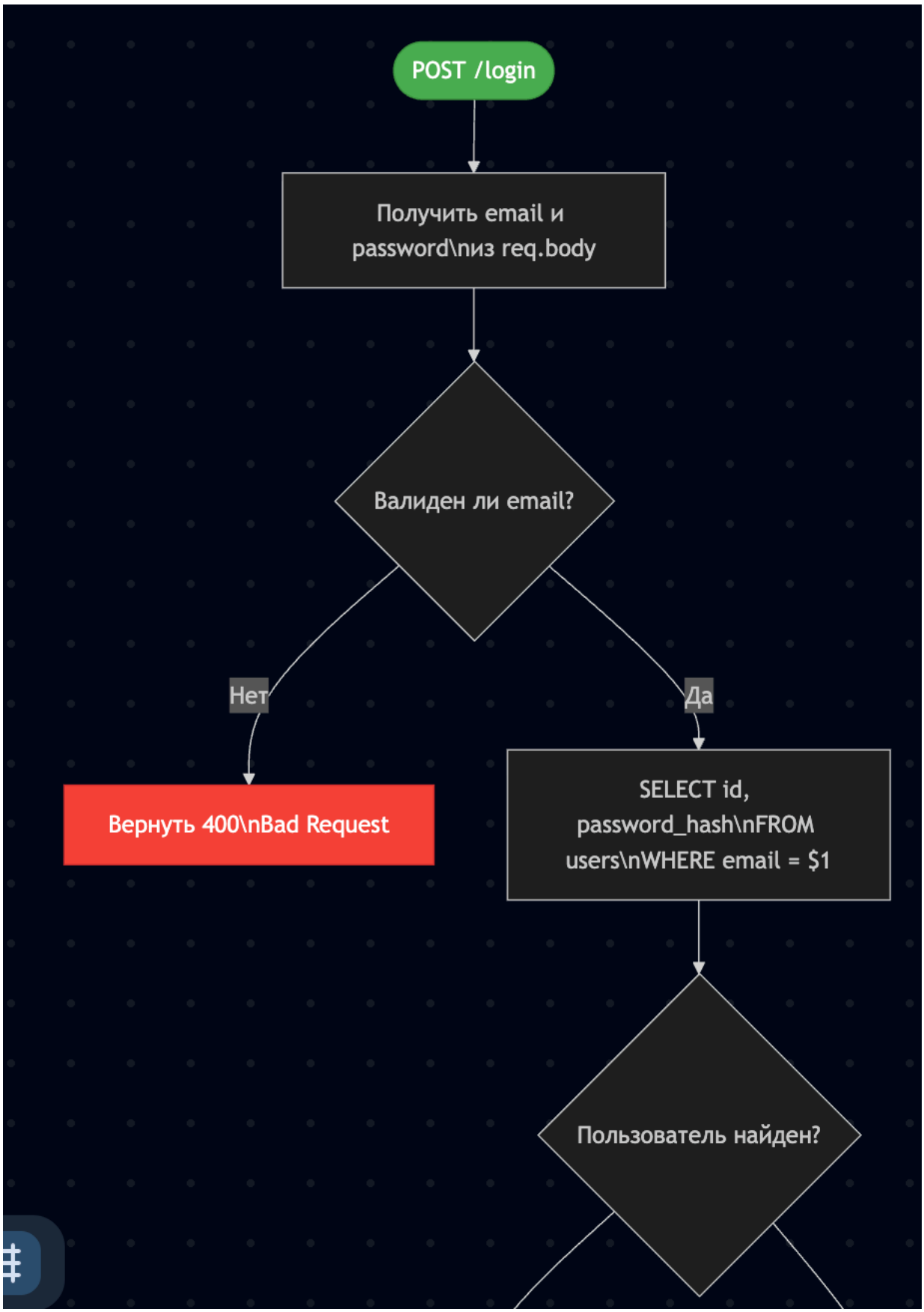

Рисунок 9 – Алгоритм работы ключевых модулей 1

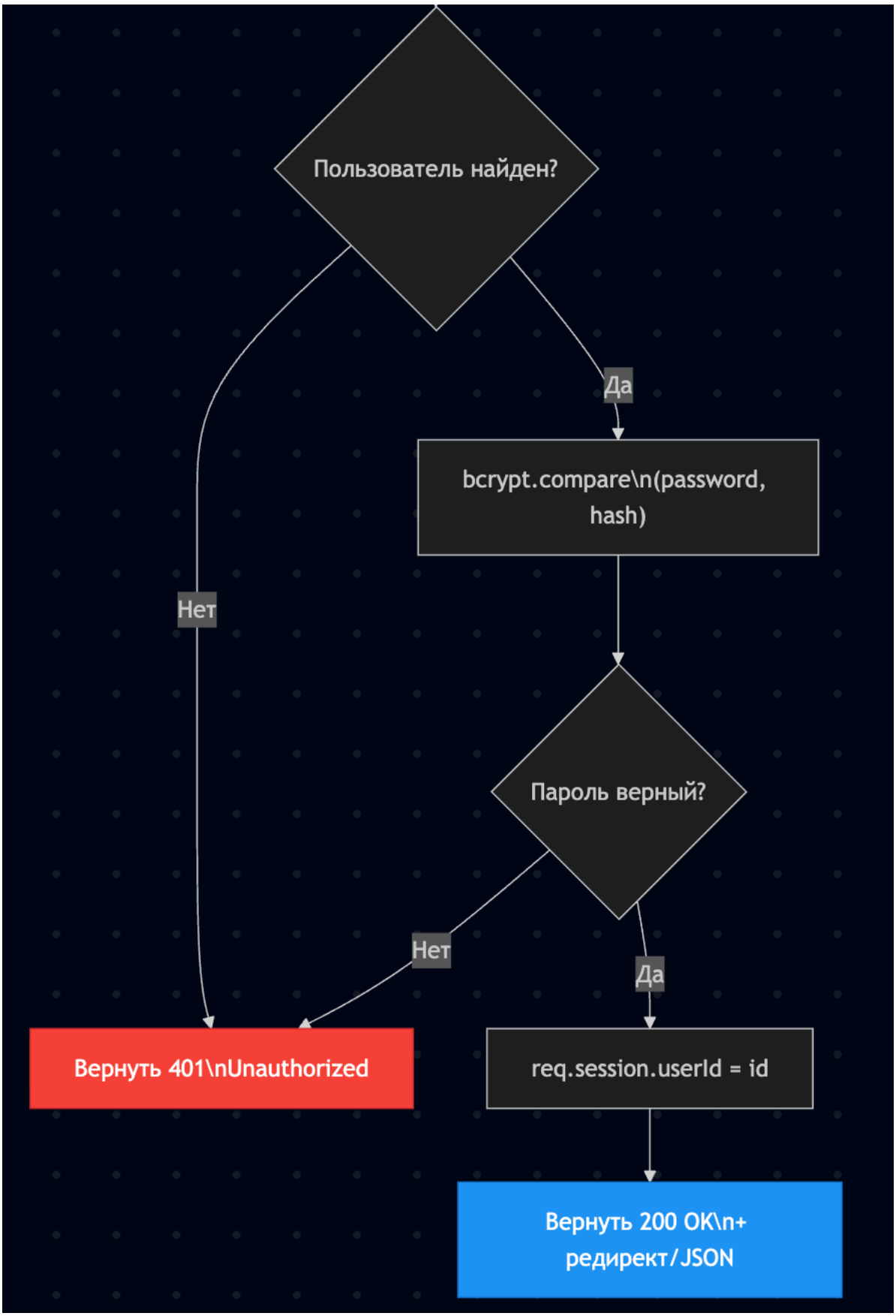

Рисунок 10 – Алгоритм работы ключевых модулей 2

- Регистрация (POST /register)
    1. Валидация полей.

2. Хеширование пароля bcrypt.hash(password, saltRounds).
3. INSERT into users (...).

- Публикация объявления (CREATE listing)
  1. form-data отправлена → Multer сохраняет файлы (photos) во временную папку/public/images.
  2. Валидировать входные поля (title, price, address).
  3. INSERT в listings с полями, включая JSON списка путей к фото.
  4. После вставки emit Socket.IO событие listing:created с payload (id, title, price).
- Бронирование (transactional)
  1. Клиент → POST /bookings (listing_id, user_id, dates).
  2. Начинается транзакция в БД: BEGIN.
  3. Проверка доступности даты (SELECT ... FOR UPDATE или SELECT + логика).
  4. INSERT booking со статусом pending (или confirmed).
  5. COMMIT или ROLLBACK при ошибке.
  6. По успешному бронированию — notifyUser(ownerId, payload) и emailUser(участники,...).
- Чат / Сообщения
  1. Клиент подключается к Socket.IO; сервер при подключении привязывает сокет к комнате user_<id> (если аутентифицирован).
  2. При отправке сообщения: emit message:send с {to, text}.
  3. Сервер принимает: INSERT в messages (sender_id, receiver_id, text).
  4. Сервер делает io.to('user_<receiver>').emit('message:new', payload).
  5. Если получатель оффлайн — хранится в БД; при следующем подключении присылается unreadCount.

3. Диаграмма состояний системы (текстовый вариант и график)

   Состояния:

- S0: Инициализация (Initialization) — старт сервера, подключение к БД, настройка mailer.

- S1: Ожидание (Idle / Listening) — сервер принимает HTTP/WebSocket соединения.
- S2: Аутентифицированный пользователь (Authenticated Session) — после успешного входа у клиента есть сессия.
- S3: В процессе транзакции (InTransaction) — бронирование/оплата (операция требующая ACID).
- S4: В реальном чате (ChatActive) — активный WebSocket обмен.
- S5: Отключение/Шатдаун (Shutdown).

Условия перехода:

- S0 → S1: успешная инициализация компонентов (DB connected, server.listen).
- S1 → S2: POST /login успех или регистрация с автологином.
- S2 → S3: пользователь инициирует бронирование/оплату; при commit возвращается в S2; при rollback — в S2 с ошибкой.
- S2 → S4: пользователь открывает чат/устанавливает WebSocket соединение.
- S4 → S2: пользователь закрывает чат/соединение.
- Любое состояние → S5: получение SIGINT/SIGTERM или критическая ошибка.

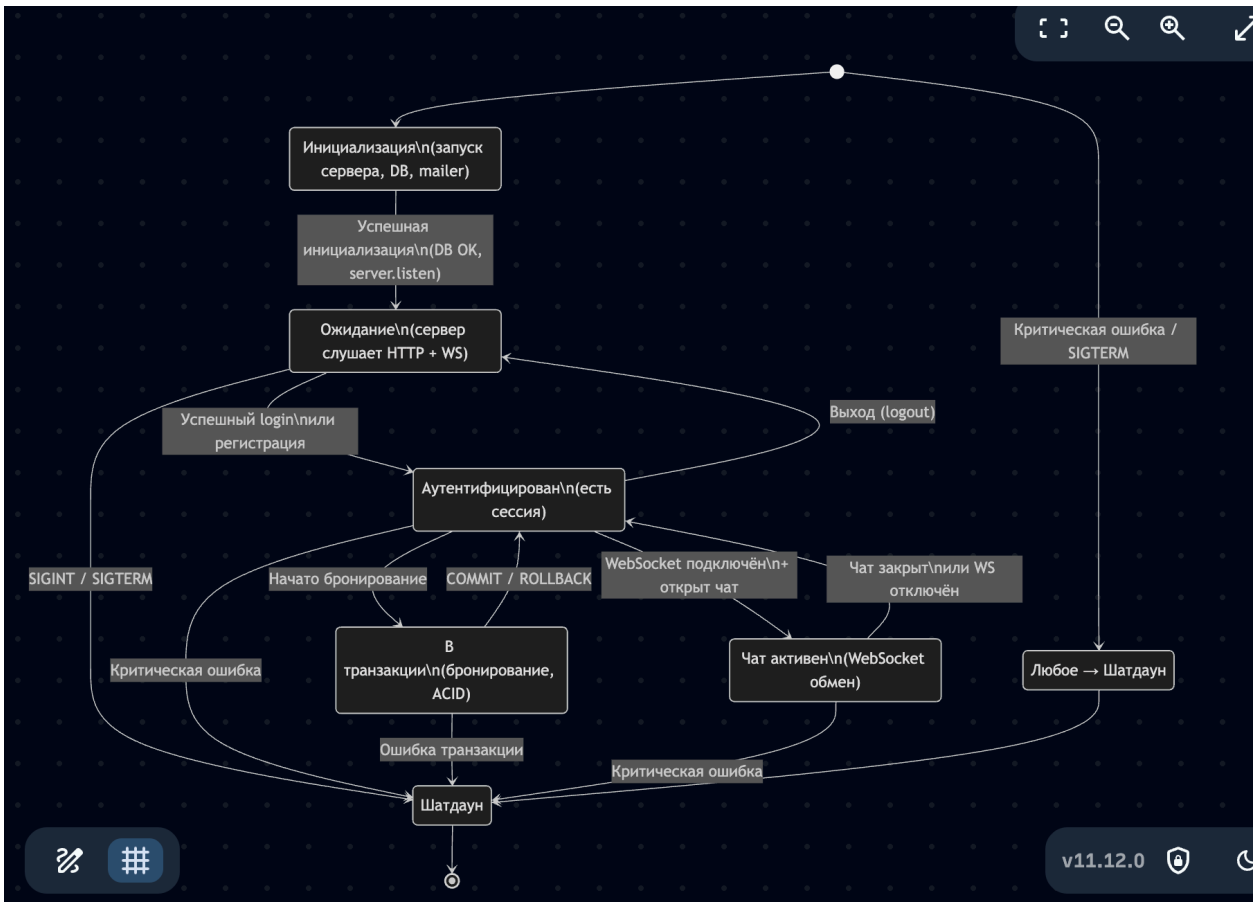

Рисунок 10 – Алгоритм работы ключевых модулей 2

4. Диаграммы последовательности (sequence diagrams — текст/псевдо)

- Сценарий: Login

  Participant: Browser, Express(Server), DB, SessionStore

  Sequence:

  Browser -> Server: POST /login {email,password}

  Server -> DB: SELECT id, password_hash FROM users WHERE email=...

  DB --> Server: row (or empty)

  Server: bcrypt.compare(password, hash)

  alt success

  Server -> SessionStore: create session (userId)

  Server --> Browser: 200 OK + Set-Cookie

  else fail

  Server --> Browser: 401 Unauthorized

  end

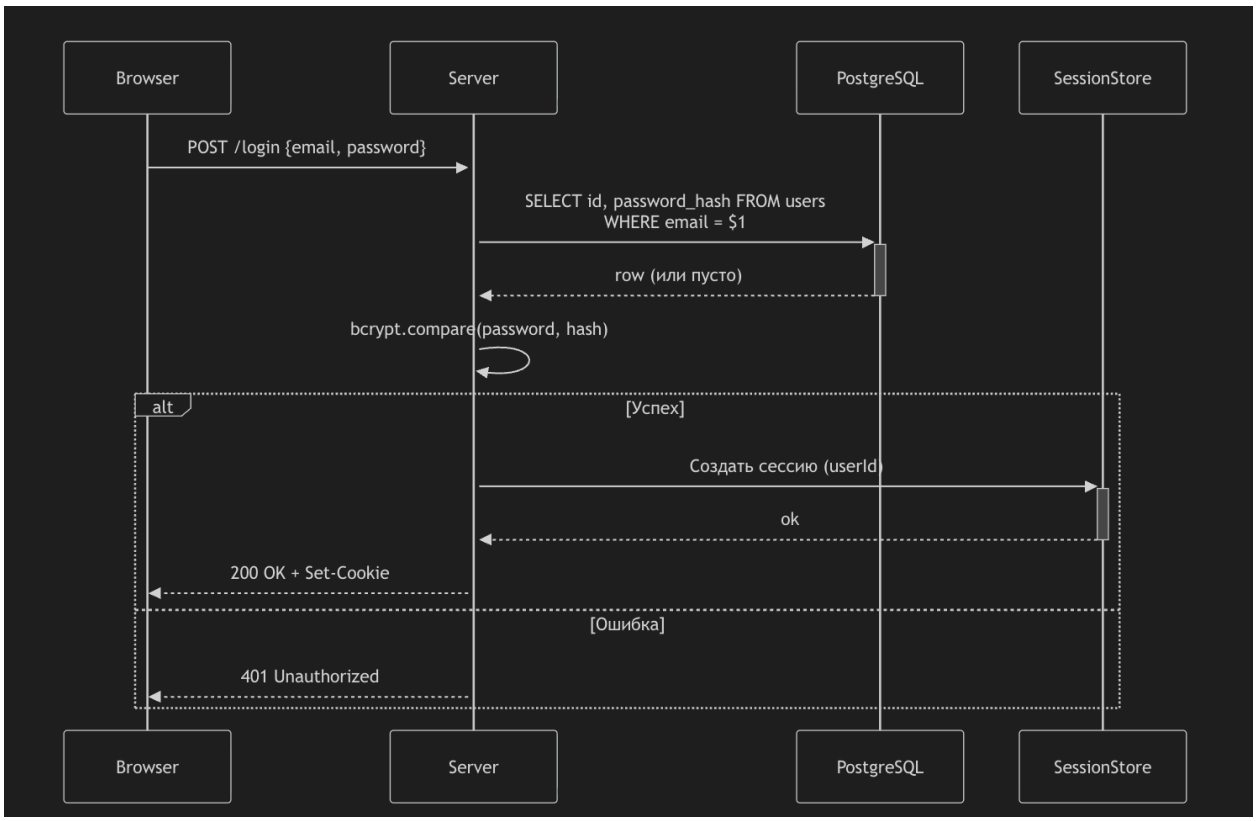

Рисунок 11 – Сценарий логин

- Сценарий: Create Listing

  Browser -> Server: POST /listings (multipart + photos)

  Server -> Multer: store files -> FS paths

  Server -> DB: INSERT INTO listings (...)

  DB --> Server: inserted id

  Server -> Socket.IO: io.emit('listing:created', {...})

  Server --> Browser: 201 Created {id}

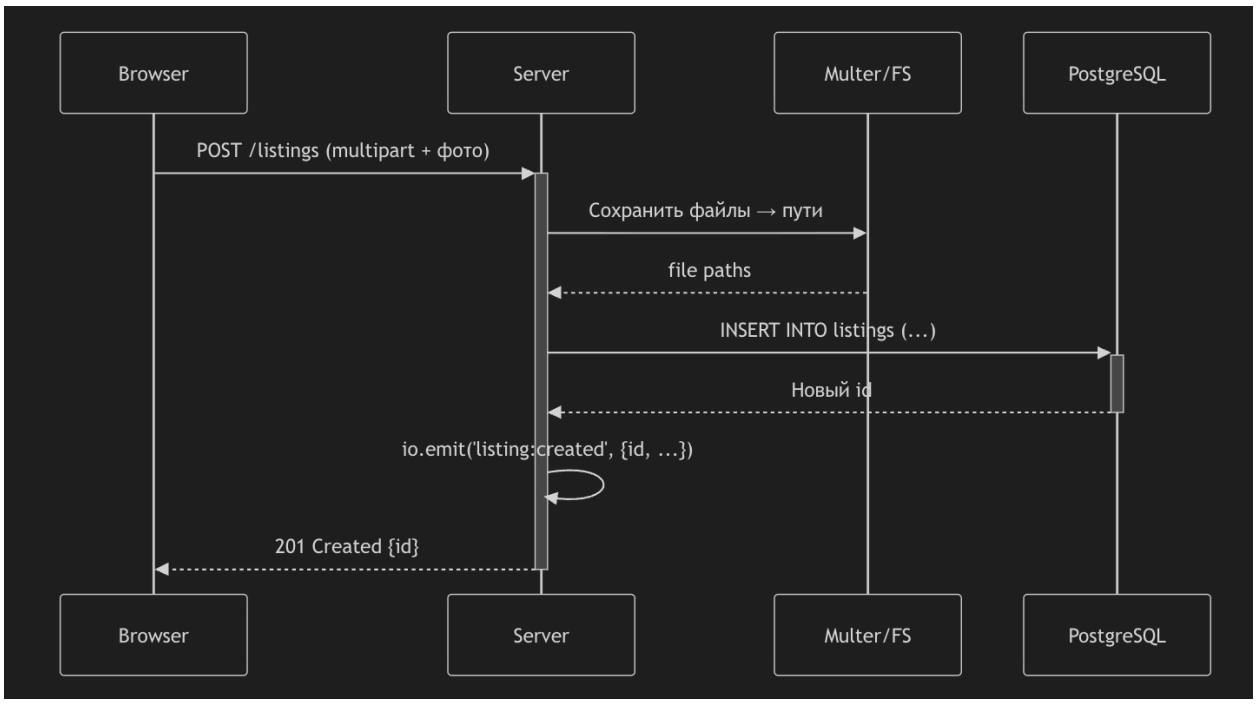

Рисунок 12 – Сценарий создания объявления

- Сценарий: Send Message (real-time)

  SenderBrowser -> Socket.IO(Server): emit 'message:send' {to, text}

  Server -> DB: INSERT INTO messages (sender, receiver, text)

  DB --> Server: ok

  Server -> Socket.IO: io.to('user_<receiver>').emit('message:new', payload)

  alt receiver online

  ReceiverBrowser <-- Server: 'message:new'

  else

  Server: store unread flag, maybe send emailUser(receiver,...)

  end

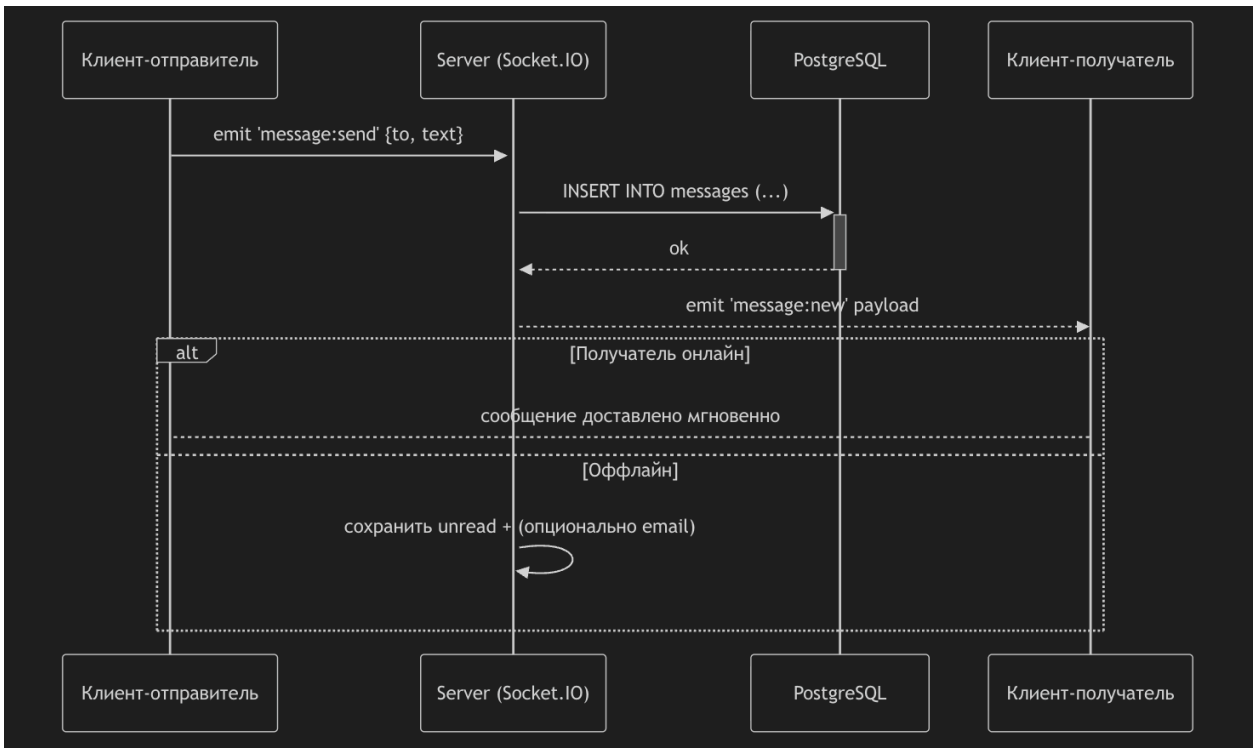

Рисунок 13 – Сценарий отправки сообщения

- Сценарий: Booking (transaction)

  Browser -> Server: POST /bookings {listing_id, dates}

  Server -> DB: BEGIN

  Server -> DB: SELECT availability FOR UPDATE

  alt available

  Server -> DB: INSERT booking

  Server -> DB: COMMIT

  Server -> Socket.IO: notify owner and renter

  Server --> Browser: 200 {booking_id}

  else

  Server -> DB: ROLLBACK

  Server --> Browser: 409 Conflict (not available)

  end

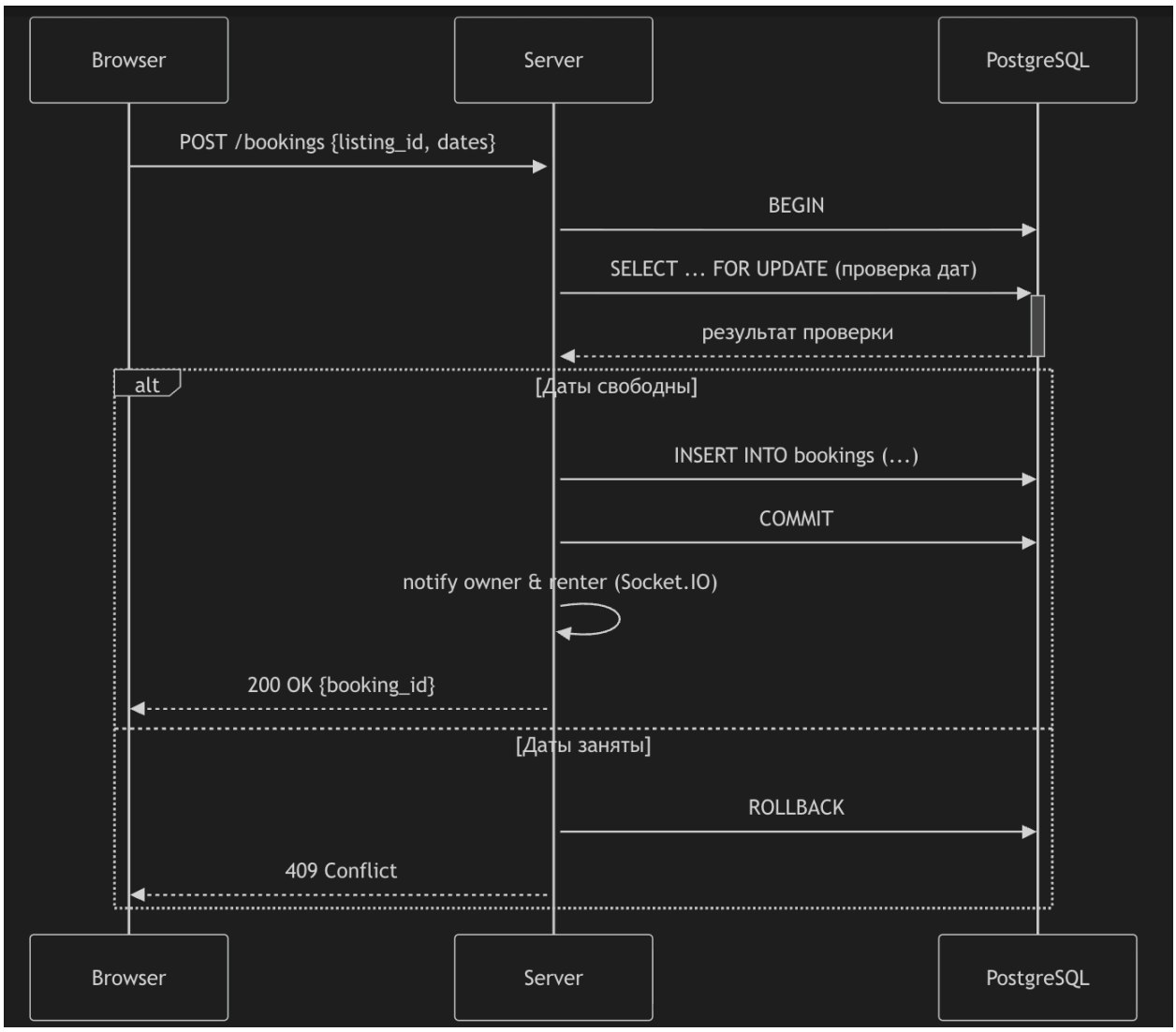

Рисунок 14 – Сценарий бронирования

5. Программные интерфейсы (API и события Socket.IO)

- HTTP Endpoints
    - POST /login
        - Вход: {email, password}
        - Выход: 200 + Set-Cookie / 401
    - POST /register
        - Вход: {email, password, name, ...}
        - Выход: 201 / 400
    - GET /listings
        - Вход: query params (search, filters)
        - Выход: 200 JSON [listings]

- POST /listings (auth required)
    - Вход: multipart/form-data (title, price, photos, amenities, lat, lng, ...)
    - Выход: 201 {id}
- GET /listing/:id
    - Выход: 200 {listing}
- POST /bookings (auth)
    - Вход: {listing_id, check_in, check_out}
    - Выход: 200 {booking} или 409
- GET /messages (auth)
    - Выход: 200 [messages]
- POST /logout
    - Выход: 200 (session destroyed)

Для каждого endpoint: использовать параметризованные SQL запросы через pool.query(sql, params).

- Socket.IO события (сервер ↔ клиент)
    - Клиент → Сервер:
        - auth (при соединении) — payload: {sessionId or token}
        - message:send — payload: {to, text, threadId?}
        - typing — payload: {to, isTyping}
    - Сервер → Клиент:
        - message:new — payload: {from, text, created_at, id}
        - notify — payload: {type, data} (в коде реализовано: notifyUser(io, userId, payload))
        - listing:created, booking:status, unreadCount:update
- Внутренние модули / функции (интерфейсы)
    - DB layer (рекомендуется вынести) — функции:
        - getUserByEmail(email) -> Promise<UserRow|null>
        - createUser(data) -> Promise<id>

        - createListing(data) -> Promise<id>
        - createBooking(bookingData) -> Promise<bookingId> (в транзакции)
        - addMessage(senderId, receiverId, text) -> Promise<messageId>
        - getUnreadCount(userId) -> Promise<number>
    - Notification layer:
        - notifyUser(io, userId, payload) — отправляет через socket room user_<id>.
        - emailUser(userId, subject, text) — ищет email по userId и отправляет письмо, использует mailer если настроен.
    - Storage layer:
        - storePhoto(file) -> pathOrUrl — сохраняет файл и возвращает путь/URL.
    - Auth/session:
        - ensureAuthenticated(req, res, next) — middleware, проверяет req.session.userId.

6. Рекомендации по реализации и тестированию

- Сделать слой доступа к данным (DAO) вместо прямых SQL в route handlers — улучшит тестируемость.
- Для бронирования использовать транзакции и SELECT ... FOR UPDATE для предотвращения race conditions.
- Логи и мониторинг (пометить critical errors в Sentry/Logstash).
- Для production: использовать HTTPS, session store (redis) вместо памяти, масштабирование Socket.IO (adapter Redis) при нескольких инстансах.
- Добавить unit/integration тесты для основных бизнес-функций (auth, booking, messaging).

ЗАКЛЮЧЕНИЕ

В ходе выполненной курсовой работы была разработана полноценная веб-платформа для аренды жилья с разделением ролей «арендатор» и «арендодатель», реализованная на современном стеке Node.js + Express + PostgreSQL + Socket.IO и интегрированная с Яндекс.Картами. Полностью реализованы все запланированные функциональные блоки: аутентификация и сессии, личный кабинет, создание и управление объявлениями с загрузкой фотографий, поиск и фильтрация, система бронирований с проверкой пересечения дат и мини-календарём, чаты и уведомления в реальном времени, отзывы и рейтинг, избранное, история просмотров, мультиязычность (русский/английский) и мобильная адаптация интерфейса.

Разработанная архитектура соответствует принципам многослойного проектирования, обеспечивает безопасность данных (хеширование паролей, сессии, защита от основных веб-уязвимостей), масштабируемость и удобство дальнейшего расширения. Применение WebSocket через Socket.IO позволило реализовать мгновенные уведомления и живой чат без перезагрузки страниц, что существенно повысило удобство взаимодействия пользователей.

Полученный прототип полностью соответствует исходному техническому заданию, демонстрирует практическое владение современными веб-технологиями и может служить основой для реального коммерческого сервиса аренды жилья, а также использоваться в учебных целях как пример полноценного full-stack-приложения на JavaScript-экосистеме.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 34.602-2020. Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы. М.: Стандартинформ, 2020.
2. ГОСТ Р 54869-2011. Проектирование программного обеспечения. Требования к содержанию и оформлению документов. М.: Стандартинформ, 2012.
3. Документация Node.js v20.x (официальная). URL: https://nodejs.org/docs/latest-v20.x/api/ (дата обращения: 25.11.2025).
4. Официальная документация Яндекс.Карт API для JavaScript. URL: https://yandex.ru/dev/maps/jsapi/ (дата обращения: 25.11.2025).
5. Официальная документация Express.js 4.x. URL: https://expressjs.com/ru/ (дата обращения: 25.11.2025).
6. Официальная документация Socket.IO. URL: https://socket.io/docs/v4/ (дата обращения: 25.11.2025).
7. Таненбаум Э., Бос Х. Современные операционные системы. 4-е изд. СПб.: Питер, 2015. 1120 с.
8. Фаулер М. Рефакторинг: улучшение существующего кода. 2-е изд. М.: ДМК Пресс, 2019. 496 с.
9. Хавербеке М. Выразительный JavaScript: Современное веб-программирование. 3-е изд. СПб.: Питер, 2020. 512 с.
10. Хоган Б. Проектирование веб-API на основе REST. М.: ДМК Пресс, 2019. 352 с.

ПРИЛОЖЕНИЕ А

```
// Ленивая инициализация дополнительных колонок в listings
let listingsColumnsEnsured = false;
async function ensureListingExtraColumns() {
  if (listingsColumnsEnsured) return;
  try {
    await pool.query(
      `ALTER TABLE listings
        ADD COLUMN IF NOT EXISTS rooms INTEGER,
        ADD COLUMN IF NOT EXISTS housing_type VARCHAR(20),
        ADD COLUMN IF NOT EXISTS district VARCHAR(100),
        ADD COLUMN IF NOT EXISTS amenities TEXT,
        ADD COLUMN IF NOT EXISTS lat DOUBLE PRECISION,
        ADD COLUMN IF NOT EXISTS lng DOUBLE PRECISION,
        ADD COLUMN IF NOT EXISTS photos TEXT`
    );
  } catch (_) {}
  listingsColumnsEnsured = true;
}

// Сессии
app.use(
  session({
    secret: "mysecret",
    resave: false,
    saveUninitialized: false,
  })
);
```

```
// Middleware для подсчёта непрочитанных сообщений (перемещён выше
роутов)
app.use(async (req, res, next) => {
  if (req.session.userId) {
    const unread = await pool.query(
      `SELECT COUNT(*) FROM messages WHERE receiver_id = $1 AND
is_read = false AND text <> ''`,
      [req.session.userId]
    );
    res.locals.unreadCount = parseInt(unread.rows[0].count, 10);
  } else {
    res.locals.unreadCount = 0;
  }
  next();
});

// Парсинг форм
app.use(express.urlencoded({ extended: true }));
app.use(express.json());

// Папка для статики
app.use(express.static(path.join(__dirname, "public")));

// Папка для html
app.set("views", path.join(__dirname, "views"));
app.engine("html", require("ejs").renderFile);
app.set("view engine", "html");

const uploadDir = path.join(__dirname, "public", "images");
if (!fs.existsSync(uploadDir)) fs.mkdirSync(uploadDir, { recursive: true });
```

```
const storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, uploadDir),
  filename: (req, file, cb) => {
    const ext = path.extname(file.originalname);
    cb(null, Date.now() + ext);
  },
});
const upload = multer({ storage });
const uploadMultiple = multer({ storage }).array("photos", 10); // максимум 10 фото
const uploadFields = multer({ storage }).fields([
  { name: "photos", maxCount: 10 },
  { name: "additional_photos", maxCount: 10 },
]);

// Главная страница

// Ленивая инициализация таблицы истории просмотров объявлений (глобально)
let listingViewsTableEnsured = false;
async function ensureListingViewsTable() {
  if (listingViewsTableEnsured) return;
  try {
    await pool.query(`
      CREATE TABLE IF NOT EXISTS listing_views (
        id SERIAL PRIMARY KEY,
        listing_id INTEGER NOT NULL REFERENCES listings(id) ON DELETE CASCADE,
        viewer_id INTEGER REFERENCES users(id) ON DELETE SET NULL,
        ip INET,
```

```
        user_agent TEXT,
        created_at TIMESTAMP DEFAULT NOW()
      );
      CREATE INDEX IF NOT EXISTS idx_listing_views_listing ON listing_views(listing_id);
      CREATE INDEX IF NOT EXISTS idx_listing_views_viewer ON listing_views(viewer_id);
      CREATE INDEX IF NOT EXISTS idx_listing_views_created ON listing_views(created_at);
    `);
  } catch (_) {}
  listingViewsTableEnsured = true;
}
```